FIPILOT

AI-POWERED CV-TO-INTERVIEW SYSTEM

Graduation Project Report

Software Engineering and Artificial Intelligence Project

Quy Nhon, August 2026

Acknowledgement

This report records the design and implementation of FiPilot as a graduation software project. The work combines document processing, web application development, language-model integration, knowledge retrieval, interview orchestration, speech interaction, persistence, and software verification. The authors acknowledge the guidance of lecturers, reviewers, and collaborators who supported the project. Personal names are intentionally omitted from this repository edition so that they can be inserted by the submitting team in the institutional cover and approval pages.

Abstract

FiPilot is a web-based system that converts a PDF or DOCX Resume into a structured Candidate Profile and uses that profile to conduct a personalized technical interview. The implemented workflow validates the uploaded document, performs native extraction with OCR fallback when necessary, requests a schema-constrained profile from Gemini, normalizes the result, and persists it under the authenticated user. The interview service then retrieves role-relevant knowledge from the local Knowledge catalog, creates an Interview Plan, generates questions, evaluates answers, applies deterministic continuation and difficulty rules, stores the session, and produces a final report.

The system supports both text and voice interaction. Text answers use authenticated REST operations. Voice sessions use an authenticated WebSocket, voice activity detection, speech-to-text, the shared answer-processing service, and streamed text-to-speech. The frontend is implemented with React and TypeScript; the active backend is a FastAPI modular monolith; persistence is provided through repository adapters; Firebase supplies identity verification; and Vertex AI Gemini supplies the language operations used by the application.

This document is an as-built report. It explains only components connected to the application workflow and avoids presenting research-only alternatives as system features. Software validation for the inspected revision completed 286 backend tests and 119 frontend tests, together with backend compilation, frontend type checking, linting, and a production frontend build. These results verify software behavior covered by the test suites; they are not a claim that automated interview scores replace professional human judgment.

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CV	Curriculum Vitae / Resume
DOCX	Office Open XML document format
ETag	HTTP entity tag
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
VAD	Voice Activity Detection
WebSocket	Full-duplex communication protocol over one connection

Contents

List of Figures

List of Tables

Chapter I

PROJECT INTRODUCTION

I.1 Overview

I.1.1 Project information

FiPilot is an AI-assisted web application for technical interview practice. Its defining input is a candidate Resume, and its defining output is a completed interview record with structured feedback. The application does not operate as a general conversation page. It maintains authenticated resources, a Candidate Profile, an Interview Plan, ordered turns, answer evaluations, adaptive decisions, and a final report.

Table I.1: Project overview information.

Item	Description
Project name	FiPilot
Product type	Web-based CV-to-interview application
Primary users	Candidates practising information-technology interviews
Input	One authenticated PDF or DOCX Resume and subsequent text or voice answers
Core output	Structured Candidate Profile, interview session, per-turn feedback, and final report
Frontend	React, TypeScript, Vite, React Router, TanStack Query, and Zustand
Backend	FastAPI gateway, application services, interview orchestrator, and infrastructure adapters
Language service	Vertex AI Gemini accessed through the backend integration layer
Default knowledge retrieval	Deterministic weighted lexical retrieval over the local Knowledge catalog
Identity	Firebase Authentication with server-side token verification
Persistence	SQLite or Firestore through a common repository contract
Interaction channels	HTTPS REST for text workflows and WebSocket for voice sessions

I.1.2 Product overview

The user signs in, uploads a Resume, and receives a structured Candidate Profile. The application uses this committed profile when preparing an interview. A local knowledge retriever selects technical topics related to the candidate's role, specialization, skills, and evidence. The planning service combines the profile, configuration, and retrieved context to produce an Interview Plan. The question service generates one question at a time from the current plan round. After the candidate answers, the evaluation service produces structured scores and feedback, and a deterministic decision service chooses whether to ask a follow-up, adjust difficulty, continue the plan, or complete the session.

The text and voice experiences share the same interview state. In text mode, answers arrive through an authenticated REST request. In voice mode, PCM audio reaches an authenticated WebSocket; speech components detect voice activity and produce a transcript; the transcript then enters the same answer service used by text mode. This shared boundary prevents the two channels from developing different scoring and progression rules.

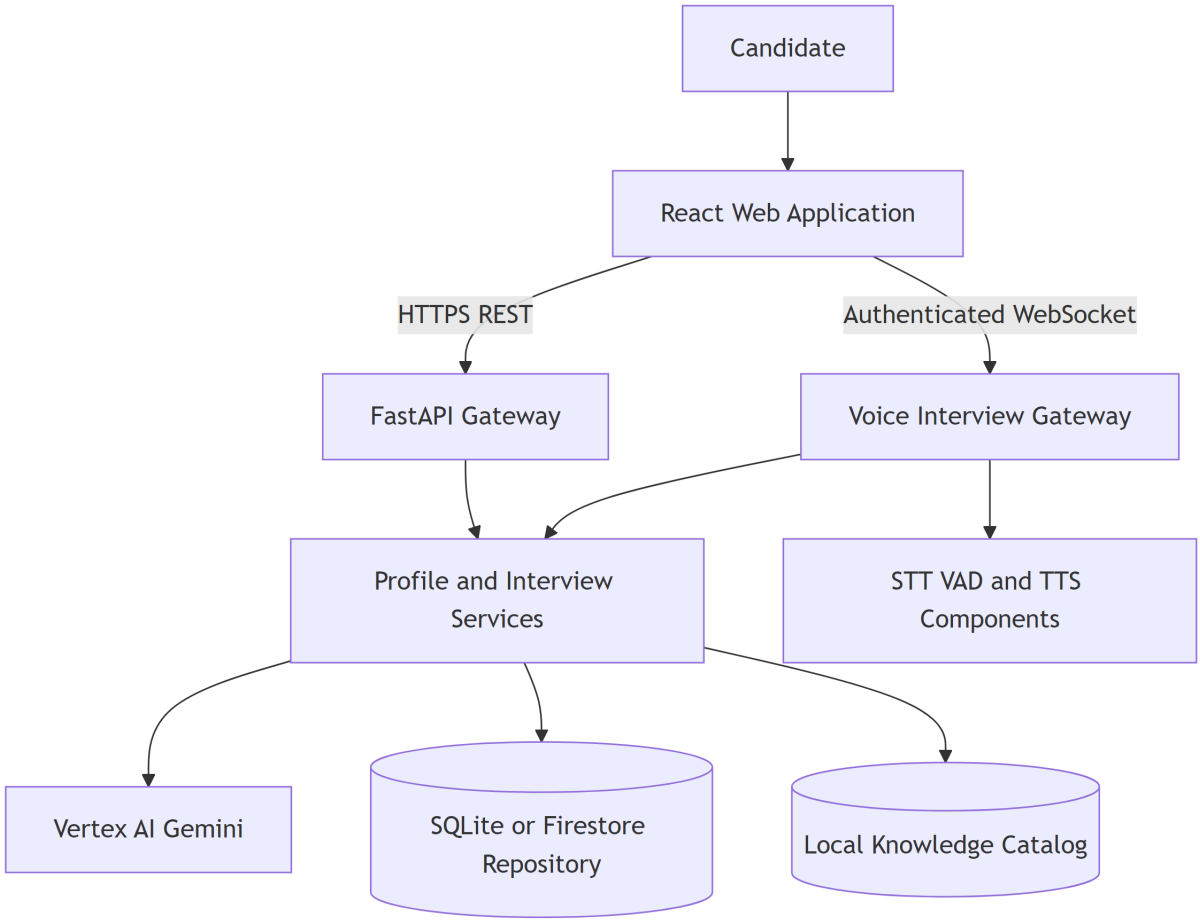

Figure I.1: FiPilot system context and connected external boundaries.

I.2 Project context

Technical interviews require candidates to connect conceptual knowledge with concrete experience. A static list of questions cannot adapt to a Resume, while an unconstrained chat cannot reliably provide ownership, state recovery, repeatable data contracts, or a durable report. Resume documents also vary in layout, format, text quality, and completeness. The application therefore needs both flexible language interpretation and deterministic software controls.

FiPilot addresses this context through a staged workflow. Document bytes are validated before extraction. Language output is accepted only through defined schemas. Knowledge selection is performed before planning. Interview state is persisted rather than left in a browser conversation. User identity is resolved by the server, and every candidate/session/report operation is scoped to that identity. These controls allow the application to use language models without delegating authentication, persistence, or workflow authority to them.

I.3 Objectives

The implemented objectives are:

1. accept one PDF or DOCX Resume, reject invalid structures, extract native text, and use OCR when a PDF does not contain enough readable text;
2. transform extracted text into the canonical Candidate Profile used by the application;
3. calculate structured profile-readiness information for the user interface;
4. select relevant technical topics from the local Knowledge catalog using deterministic lexical scoring;
5. create a structured Interview Plan and generate candidate-specific questions;
6. evaluate text or transcribed voice answers and apply deterministic continuation rules;
7. persist interview history and generate a final report;
8. protect user resources through Firebase identity verification and ownership-scoped repository operations; and
9. provide a responsive React interface for upload, profile review, text interview, voice interview, history, settings, and reports.

I.4 Problem statement

The central engineering problem is a sequence of transformations with different trust levels:

$$\text{Resume} \rightarrow \text{Profile} \rightarrow \text{Plan} \rightarrow \text{Question} \rightarrow \text{Answer} \rightarrow \text{Evaluation} \rightarrow \text{Report}. \tag{I.1}$$

Document content is untrusted input. Profile fields are model-assisted interpretations that require structural validation. Retrieved knowledge is technical context and must not be confused with a candidate claim. A model-generated score is an application signal and must not directly control persistence or ownership. The architecture must preserve these distinctions throughout a multi-turn session.

A second problem is continuity. A technical interview can contain retries, browser refreshes, double submissions, slow external calls, and voice disconnections. FiPilot therefore stores sessions and turns, uses an idempotent answer-claim mechanism, and returns server state to the client. The workflow is stateful even though individual AI operations are request-response calls.

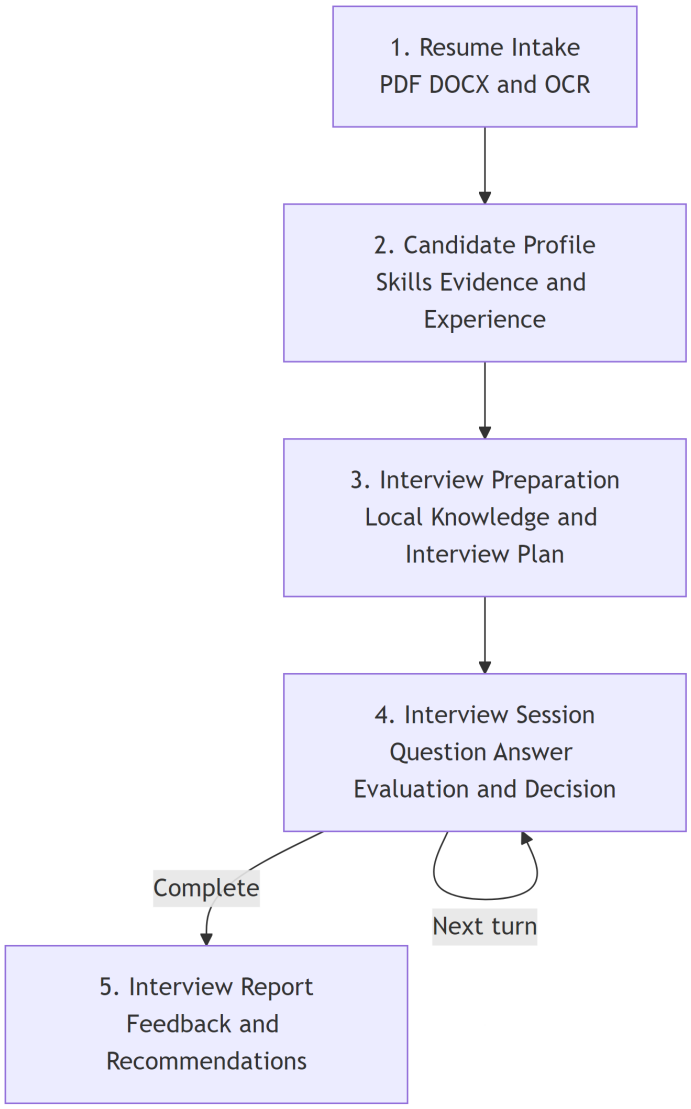

Figure I.2: Connected runtime pipeline from Resume upload to interview report.

I.5 Significance

The project demonstrates how language services can be integrated into a governed application instead of exposed as an unrestricted chat box. Candidate evidence, technical knowledge, plan structure, interview state, and evaluation output are separate domain objects. This separation makes the code testable and lets the frontend display explicit progress and errors.

The project is also significant as a full-stack integration. Resume parsing, OCR, typed model output, retrieval, orchestration, REST, WebSocket audio, authentication, two persistence adapters, and a React UI operate behind one product workflow. The resulting system is suitable for demonstrating software architecture and applied AI integration, provided that its automated feedback remains advisory.

I.6 Scope and limitations

Table I.2: Implemented scope and present limitations.

Included in the application	Present limitation
PDF and DOCX validation and extraction	Unusual layouts can still produce imperfect reading order.
OCR fallback for sparse/image PDFs	No broad real-document OCR accuracy claim is made.
Canonical Candidate Profile	Complete field-level source provenance is not exposed to the user.
Readiness information on profile responses	Start operations do not yet use readiness as a server-side rejection rule.
Local lexical knowledge retrieval	Returned planner strings do not expose a user-facing citation score.
Text and voice interview sessions	Voice quality depends on microphone, language, and speech components.
Answer evaluation and feedback	Scores are advisory and are not a substitute for a professional interviewer.
SQLite and Firestore repositories	Operational deployment state is environment-specific.
Firebase authentication and ownership checks	This report is not a security certification.
Final report generation	Historical profile consistency is constrained by the current report service behavior.

The report intentionally limits itself to the connected application. Alternative retrieval studies, disconnected prototypes, and specification-only features are outside the described system.

Chapter II

PROJECT DEVELOPMENT AND MANAGEMENT

II.1 Development organization

The repository is organized around responsibility boundaries rather than one large application module. The frontend, gateway, shared schemas, domain services, orchestrator, infrastructure adapters, speech service, tests, Knowledge catalog, documentation, and scripts form distinct work areas. This organization provides a practical work-breakdown structure even when contributors work across more than one area.

Table II.1: Repository work areas and deliverables.

Work area	Main responsibility	Repository deliverable
Product interface	Routes, forms, text/voice experiences, report views	`frontend/src`
HTTP boundary	Authentication, request validation, response contracts	`backend/gateway`
Domain contracts	Canonical Pydantic request and response models	`backend/shared/schemas`
Application services	Profile, readiness, planning, questions, evaluation, reports	`backend/services`
Workflow control	Session progression and answer decisions	`backend/orchestrator`
Infrastructure	Documents, Firebase, Gemini, speech, repositories	`backend/infrastructure`
Speech process	Optional independently started speech service	`backend/speech_service`
Knowledge	Role/topic/level Markdown documents and catalog	`Knowledge` and catalog builder
Verification	Backend and frontend behavior checks	`backend/app/tests` and frontend test files
Architecture records	ADRs, behavior specifications, deep dives	`docs`

II.2 Change management

Changes to a domain contract require reading the repository context, applicable specifications, testing seams, risk documents, and architecture decisions. Runtime source remains authoritative for what the current application does, while ADRs explain why a boundary exists. This distinction prevents a detailed proposal from being reported as completed behavior.

Canonical names are controlled centrally. For example, the Candidate Profile uses `name`, `years_experience`, `recent_role`, `specialization`, `skills`, `skill_evidence`, `projects`, `experiences`, and `education`. A change that introduces a parallel alias affects schemas, routes, repositories, tests, and UI state, so it must be reviewed as a domain change rather than a local convenience.

Small implementation changes are verified with focused tests before broader suites. Final validation includes backend tests and compilation, frontend type checking, lint, tests, and production build. No source-control commit is created automatically by report generation.

II.3 Risk management

Table II.2: Project risk matrix.

Risk	Impact	Implemented control
Unsupported Resume field	Incorrect personalization	File/text separation, schema-constrained profile output, normalization, and source reconciliation.
Malformed document	Parser failure or resource waste	Type, signature, size, container, timeout, and extraction checks.
Cross-user resource access	Privacy and integrity breach	Firebase token verification and ownership-scoped repository calls.
Invalid language response	Application contract failure	Pydantic validation and bounded integration retry behavior.
Duplicate answer submission	Repeated cost and inconsistent state	Persistent turn claim, request hash, stored result, and replay behavior.
Slow external response	Poor interaction quality	Timeouts, question prefetch, progress state, and structured errors.
Voice overload	Memory pressure	Bounded audio queues, message limits, endpoint detection, and session cleanup.
Inconsistent text/voice logic	Different evaluation outcomes	Final transcript enters the shared answer service.

Sensitive logging	Disclosure of Resume or answer content	Structured metadata logging; raw audio is not intentionally persisted.
Configuration confusion	Wrong process settings	Repository helper loads `backend/.env.local` or `BACKEND_ENV_FILE`.

II.4 Quality management

Quality controls are applied at four levels. Static contracts define types and allowed fields. Deterministic tests verify parsing, readiness, decisions, ownership, persistence, and error responses. Integration tests replace external services with controlled dependencies to exercise application behavior. Build checks ensure that the frontend and active backend packages remain executable.

Table II.3: Quality gates used by the project.

Gate	Purpose	Evidence
Schema validation	Reject malformed request/model output	Shared Pydantic models and schema tests
Ownership tests	Prevent cross-user disclosure	API and repository ownership scenarios
Repository parity	Preserve application behavior across adapters	SQLite/Firestore contract tests
Document tests	Verify valid, corrupt, mismatched, OCR, and timeout paths	Document-processing test module
Workflow tests	Verify start, answer, completion, and report behavior	Interview/orchestrator/API tests
Voice tests	Verify WebSocket, STT/TTS, interruption, and cleanup contracts	Voice service and gateway tests
Frontend tests	Verify routes, forms, errors, interaction, and accessibility behavior	React Testing Library suites
Release checks	Reject syntax/type/lint/build failures	Python compilation and frontend build commands

II.5 Documentation management

The report uses local source paths as traceability notes for implementation details. It does not reproduce environment secret values or private Resume content. Architecture figures are generated from report-owned Mermaid sources so their content can be reviewed and re-rendered. The build script regenerates figures, compiles the document, resolves the bibliography, and rejects missing references or major layout overflow.

Chapter III

EXISTING SYSTEMS AND THEORETICAL BACKGROUND

III.1 Domain overview

An interview-practice system must perform more than question generation. It needs a representation of the candidate, an interview objective, technical knowledge, controlled progression, answer interpretation, feedback, and a durable history. FiPilot treats these elements as application objects so that each operation has explicit input and output.

Traditional question banks offer predictable content but little personalization. General AI chat provides flexible language but weak resource ownership and workflow control. FiPilot combines deterministic application state with language operations. The language service interprets and generates text; the application controls files, identity, schemas, state, persistence, and error handling.

III.2 Natural language processing and Transformers

Natural language processing covers computational methods for analysing and producing human language. Resume interpretation, question wording, answer feedback, and report synthesis are NLP tasks because their meaning cannot be captured reliably by fixed field positions alone.

Modern language models commonly use the Transformer architecture. Its attention mechanism lets a token representation incorporate information from other positions in the sequence [1]. For queries Q, keys K, values V, and key dimension d_k, scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{III.1}$$

FiPilot accesses managed Gemini models through an application adapter. Transformer internals explain the language capability supplied by that external service, while FiPilot's repository is responsible for

the surrounding application controls.

III.3 Structured language output

Free-form text is inconvenient for a typed application. FiPilot therefore requests JSON that follows a response schema and validates it with Pydantic. The primary structured objects are Candidate Profile, Interview Plan, Interview Question, Answer Evaluation, and Interview Report. If the output cannot be validated, the application does not treat it as a domain object.

This separation improves controllability in three ways. First, routes and services receive known fields instead of parsing prose. Second, bounds such as score types and required nested objects can be checked. Third, malformed output becomes an explicit integration failure rather than silently changing interview state. Structural validity still does not guarantee factual correctness, so Resume normalization and source reconciliation remain necessary.

Table III.1: Structured language operations used by FiPilot.

Operation	Main input	Validated output	Application consumer
Resume interpretation	Extracted Resume text	Candidate Profile fields	Profile service and repository
Interview planning	Profile, configuration, retrieved topics	Interview Plan and rounds	Interview orchestrator
Question generation	Profile, current round, history	Interview Question	Current turn
Answer evaluation	Question, expected points, answer	Scores and feedback	Decision service and session
Report generation	Profile and completed turns	Interview Report	Report route and frontend

III.4 Retrieval-Augmented Generation

RAG combines a generative operation with external knowledge selected at request time [2]. In FiPilot, the local Knowledge catalog is the external source and retrieval occurs before interview planning. The planner receives a bounded list of technical topics together with the Candidate Profile and interview configuration. The Question Generator subsequently receives the planned round; it does not independently search the full catalog for each turn.

The implemented retriever is lexical. It normalizes profile strings into tokens, removes stop words, scores domain overlap, scores topic overlap, applies an exact-title bonus, sorts deterministically, and selects up to eight topics. This approach is suitable for a curated hierarchy because it is local, explainable,

fast, and stable. It can miss paraphrases that share few tokens, which is recorded as a system limitation rather than replaced in this report by an unused mechanism.

III.5 Adaptive interview state

Adaptive behavior is implemented as state plus deterministic policy. The model produces an Answer Evaluation, including a follow-up signal and score. The decision service prioritizes clarification when required. If no clarification is needed and the score reaches the configured threshold, difficulty can increase within the plan. Otherwise the session continues at the current level or completes when its question budget is reached.

This mechanism is not an unrestricted model decision. The application owns the state transition and persists it. The distinction matters because the same evaluation must lead to the same application rule in text and voice modes.

III.6 Document extraction technologies

PDF stores positioned page content and does not guarantee reading order. DOCX is a zipped document format containing paragraphs, tables, relationships, and styling. FiPilot uses dedicated parsers for these formats. A PDF with too little native text enters an OCR path in which pages are rendered and processed by RapidOCR. A DOCX path reads paragraphs and top-level tables. Both paths produce normalized text for the same profile service.

OCR is a fallback because image recognition is slower and less reliable than reading embedded text. The document layer limits size, validates file signatures, bounds processing time, and returns structured warnings/errors. These controls protect the language service from receiving an arbitrary malformed container.

III.7 Speech recognition and synthesis

Voice mode introduces three functions. Silero VAD detects voice activity and endpoints. Faster-whisper converts audio into partial and final transcripts. VieNeu-TTS synthesizes the next question into streamed audio. The gateway uses bounded queues and separates binary PCM frames from JSON control events.

The final transcript becomes a normal candidate answer. This architectural choice keeps the interview evaluator and decision service independent of the input medium. Raw audio is transient application data and is not intentionally written to the interview repository.

III.8 LLM-as-a-Judge limitations

FiPilot uses a language model to produce structured answer evaluation. LLM judges can apply a rubric consistently at application scale, but published research identifies sensitivity to prompt framing, verbosity, position, and model-family effects [3]. Accordingly, FiPilot feedback is advisory. The system stores evidence and scores for review; it does not establish an autonomous employment decision.

III.9 Technology selection summary

Table III.2: Technology choices and their system responsibilities.

Technology	Responsibility	Selection rationale
React/TypeScript	Browser routes and interaction	Typed component ecosystem and testable state
FastAPI/Pydantic	HTTP/WebSocket boundary and schemas	Explicit validation and dependency composition
Gemini via Vertex AI	Language interpretation and generation	Managed task-specific JSON generation
Local lexical retrieval	Planner knowledge context	Deterministic local ranking over curated topics
SQLite/Firestore adapters	Durable user/session/report data	Local development and configurable cloud persistence
Firebase Authentication	User identity	Server-verifiable browser identity token
RapidOCR	Image-PDF fallback	Extract text when native PDF text is insufficient
Faster-whisper, Silero, and VieNeu	Voice input/output	Streaming transcript, endpoint detection, and speech response

Chapter IV

AI METHODOLOGY AND SOLUTION

IV.1 Method overview

FiPilot uses language services inside a controlled application pipeline. The model is responsible for interpreting Resume text and producing schema-constrained content. Application code remains responsible for file validation, identity, ownership, retrieval, workflow state, retries, persistence, and report access. This division is central to the solution: probabilistic output is used where language interpretation is useful, while deterministic rules protect the product contract.

The connected AI-assisted operations are Resume profiling, interview planning, question generation, answer evaluation, and report generation. Each operation has its own prompt and response schema. The backend converts validation failures and provider failures into application errors instead of passing incomplete model output directly to the client.

IV.2 Resume understanding

IV.2.1 Document admission and extraction

The upload boundary accepts one PDF or DOCX document. The service checks the declared media type, file extension, size, and document signature before extraction. PDF extraction first uses the document's text layer. If the extracted text is too sparse, the service renders pages for OCR. DOCX extraction reads document paragraphs and tables. This sequence avoids unnecessary OCR while retaining a recovery path for scanned documents.

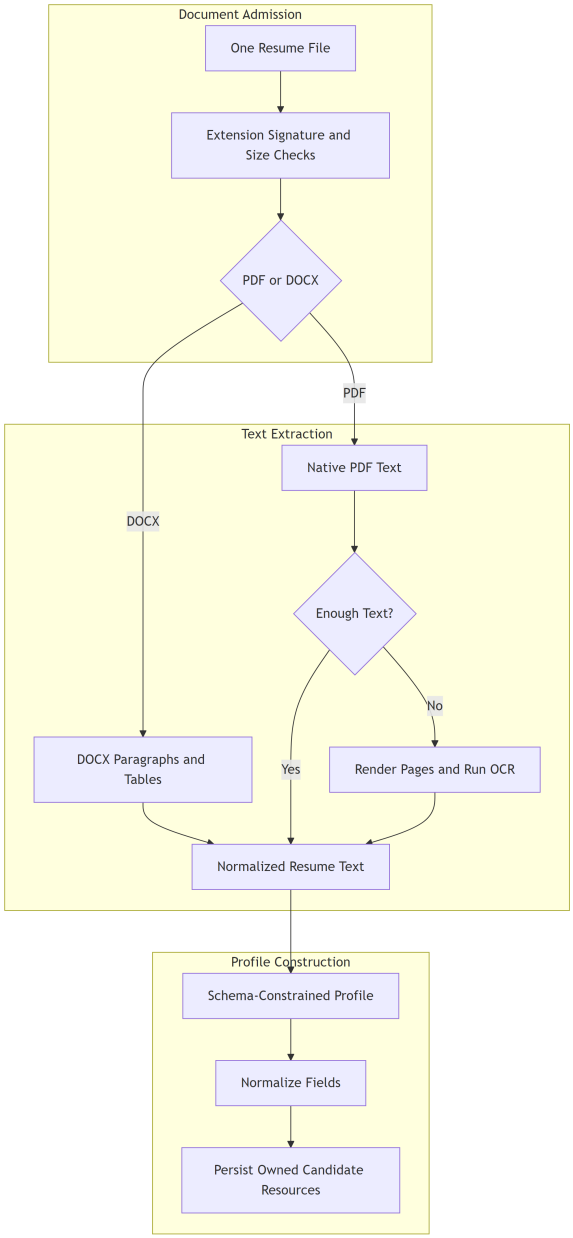

Figure IV.1: Resume validation, extraction, OCR fallback, and profile creation.

The extracted text remains untrusted. It can contain malformed characters, instructions addressed to a model, or incomplete information. The profile scanner receives a system-defined task, the Resume text as data, and a strict response schema. Pydantic validation then converts accepted output to the canonical profile type.

IV.2.2 Canonical Candidate Profile

The profile is the application representation of candidate information. It prevents later services from inventing their own field names or repeatedly parsing the original document. Its editable top-level fields are listed in table IV.1.

Table IV.1: Canonical Candidate Profile fields.

Field	Purpose
`name`	Candidate display name.
`years_experience`	Normalized experience duration when supported by the document.
`recent_role`	Most recent professional role.
`specialization`	Technical direction used by planning and retrieval.
`skills`	Normalized skill list.
`skill_evidence`	Skill-to-evidence items derived from Resume statements.
`projects`	Structured project descriptions, technologies, and role.
`experiences`	Structured employment history.
`education`	Legacy text or explicitly structured education entries.

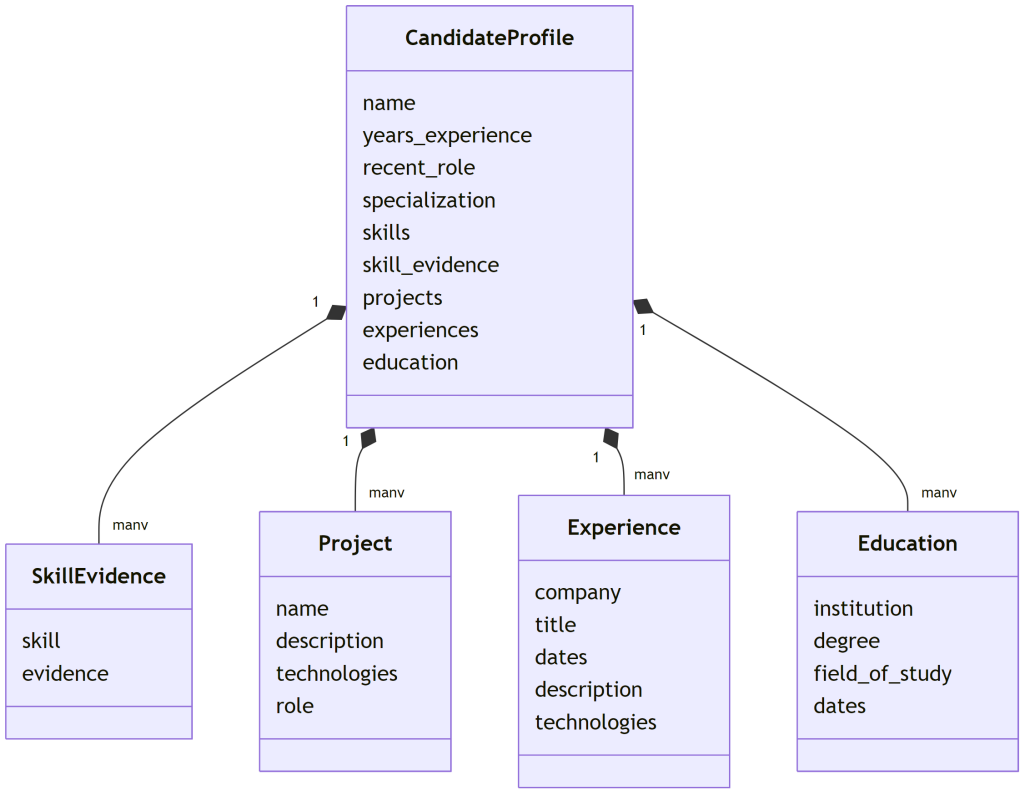

Figure IV.2: Candidate Profile as the shared contract between Resume review and interview preparation.

The profile response also carries server-generated metadata such as its identifier, version, and readiness information. These values are not model-authored editable fields. Keeping them outside the editable allowlist protects ownership, concurrency, and application decisions.

IV.3 Knowledge selection

IV.3.1 Knowledge catalog

The repository includes a local catalog of technical topics. Entries are grouped by role and contain a topic title, hierarchical path, and anchor concepts. A separate level guide describes the expected depth for candidate levels. At runtime, the catalog loader turns these source records into compact planner context; it does not expose the complete catalog to every model request.

IV.3.2 Deterministic lexical retrieval

The default retriever compares normalized query terms with catalog titles, paths, and anchors. Query terms originate from the Candidate Profile and Interview Configuration. Weighted exact and partial lexical matches produce a deterministic score, and the highest ranked eight records are supplied to the planner. Stable tie handling makes the same catalog and query repeatable.

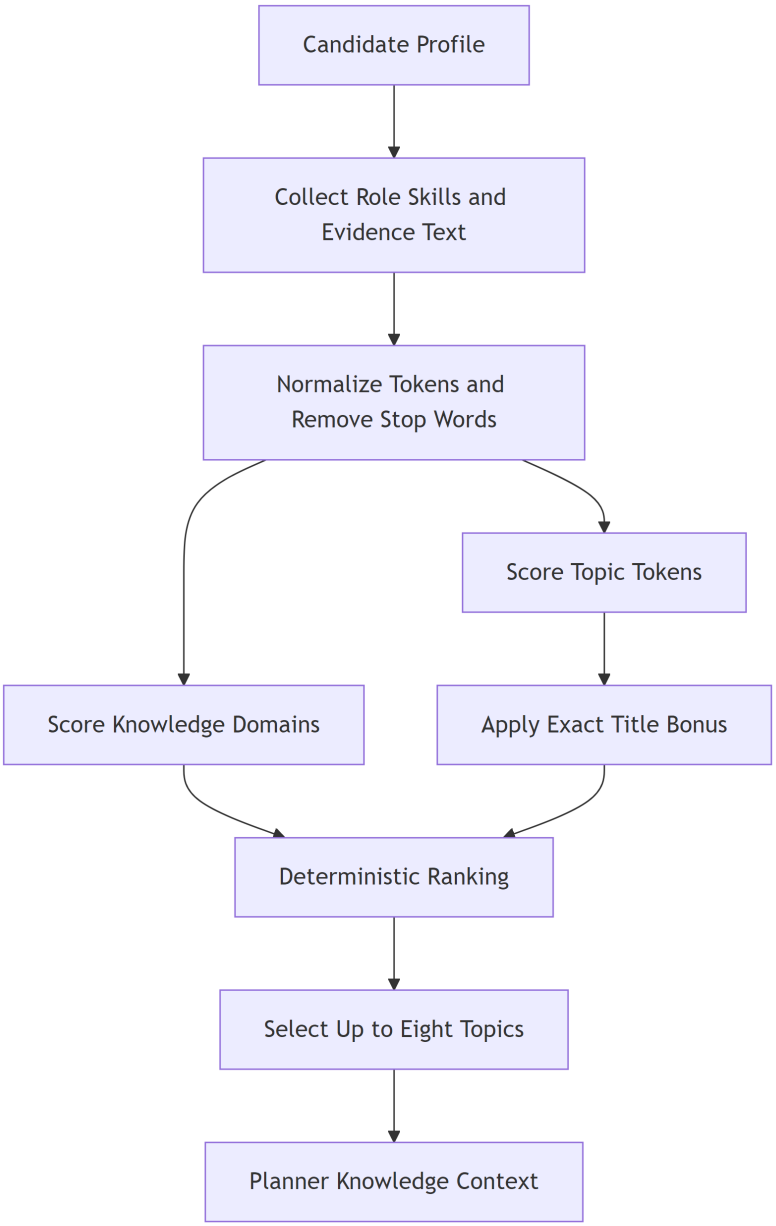

Figure IV.3: Local lexical retrieval used to curate planner context.

This design has three useful properties. First, the knowledge source can be reviewed in the repository. Second, retrieval can be tested without a network call. Third, technical context remains distinct from Candidate Profile evidence. The retrieved records guide coverage; they do not become claims about the candidate.

IV.4 Interview planning and question generation

The planner receives the committed Candidate Profile, the Interview Configuration, and curated knowledge context. It returns a structured plan containing ordered rounds and their goals. The question service then generates a question for the active round. It receives only the state needed for that turn, including prior turns and follow-up context.

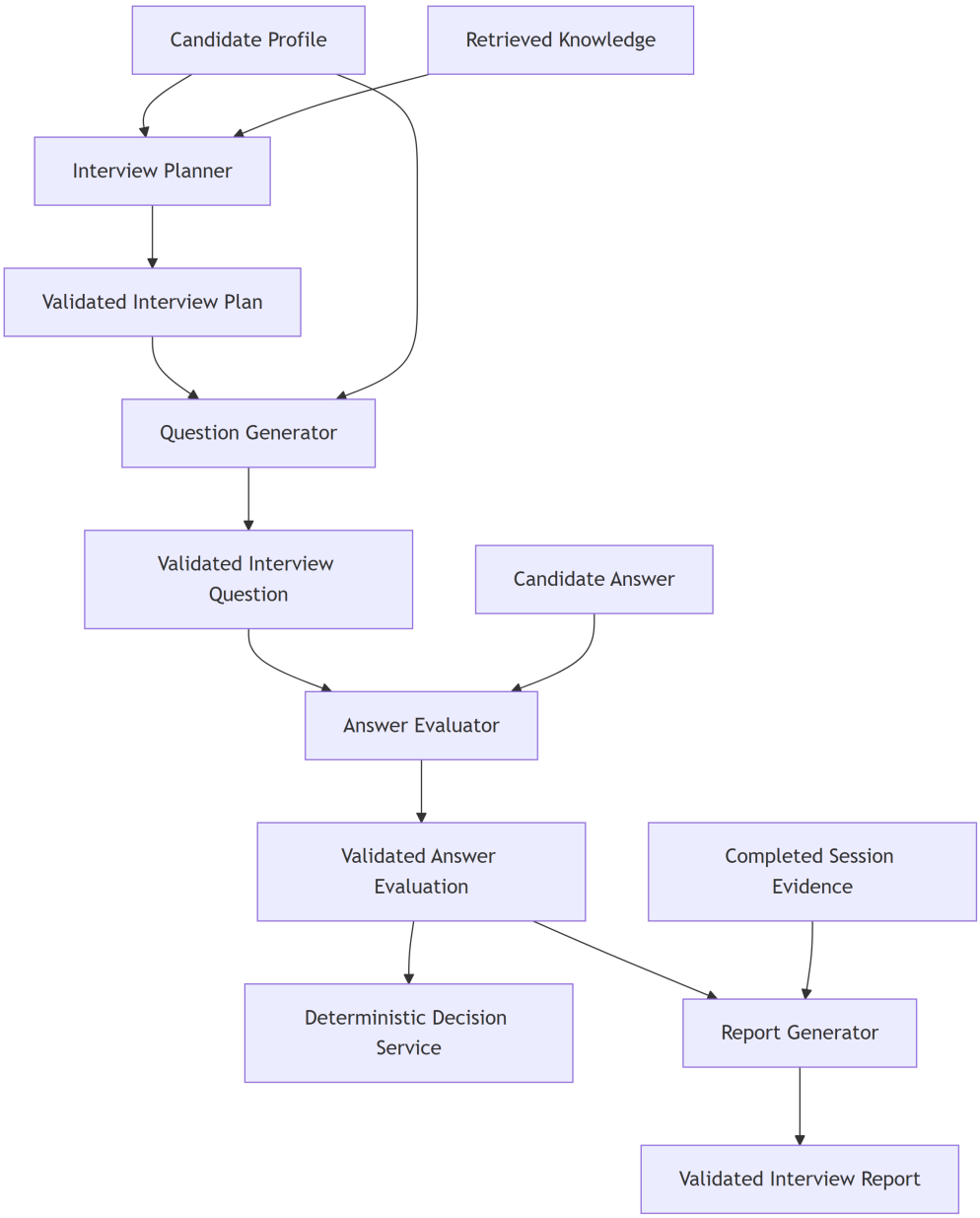

Figure IV.4: Model-assisted services and deterministic orchestration boundaries.

The plan separates session-level intent from individual wording. This means the application can persist progress, identify the current round, and request another question without asking a model to reconstruct the entire interview from free-form chat history. Question generation uses a slightly higher response variation than the profile, planner, evaluator, and report operations, while still requiring a structured response.

Table IV.2: Connected language-service operations.

Operation	Primary input	Validated output
Profile scanner	Extracted Resume text	Canonical Candidate Profile fields
Interview planner	Profile, configuration, selected knowledge	Ordered Interview Plan
Question generator	Active round and session history	One structured interview question
Answer evaluator	Question, answer, profile context, and round goal	Scores, feedback, and follow-up signal
Report generator	Completed session and associated profile data	Structured summary, strengths, gaps, and recommendations

IV.5 Answer evaluation and adaptive decisions

The evaluator assesses the submitted answer against the question and round goal. Its output contains structured dimensions and feedback rather than an unrestricted paragraph. The backend validates ranges and required fields. The evaluator can recommend a follow-up, but it does not directly mutate the session.

The decision service applies deterministic rules to the current state and accepted evaluation. It considers follow-up limits, round progress, and completion conditions. It selects the next action, after which the orchestrator persists the turn and, when required, invokes the question service for the next prompt.

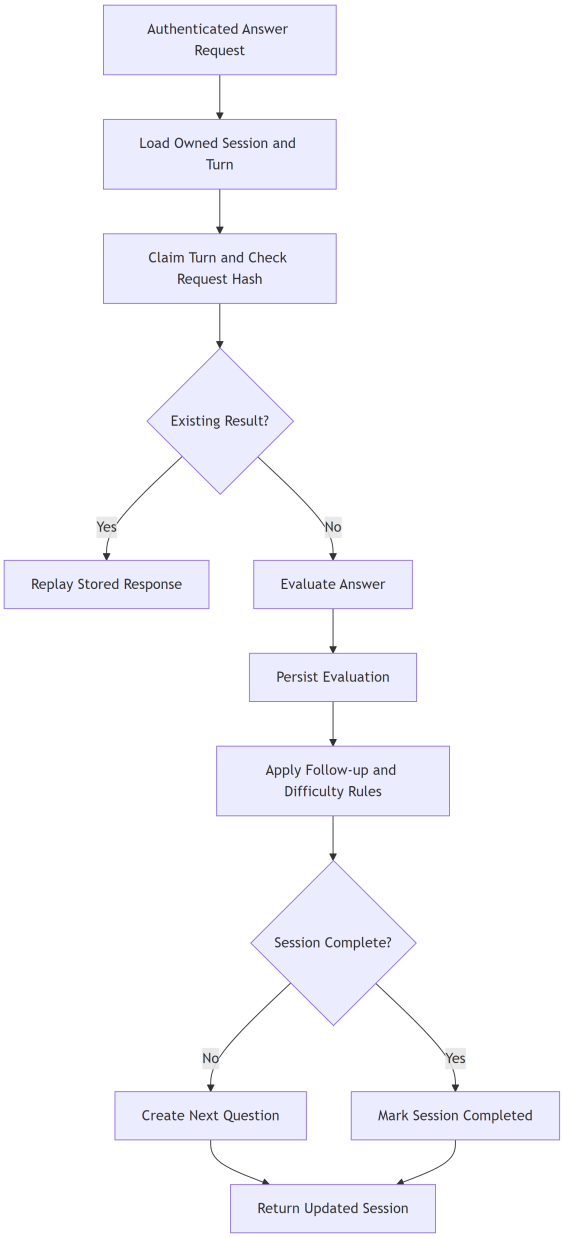

Figure IV.5: Shared answer processing for text submissions and voice transcripts.

Answer submission uses an idempotency key. The repository claim prevents concurrent repeats of the same logical answer from creating multiple turns. Once processing is complete, the stored result can be returned to a retrying client. This control is particularly important when an external language call completes after a client timeout.

IV.6 Report generation

At the end of a session, the report service collects the persisted interview state and generates a structured report. The report includes an overall assessment, strengths, improvement areas, and turn-level evidence available from the session. Reports are saved and displayed through the report route. Automated scores and suggestions remain advisory because model-based evaluation can be sensitive to wording, incomplete context, and provider behavior.

IV.7 Safety and reliability controls

The solution applies the following controls around language operations:

- typed schemas at every model boundary;
- fixed application prompts separated from user-provided document and answer data;
- bounded provider timeouts and error translation;
- repository-backed session state instead of model-owned state;
- deterministic retrieval and continuation decisions;
- authenticated, ownership-scoped access to profiles, sessions, and reports; and
- explicit UI errors instead of silent substitution when a required service fails.

These controls do not make model output infallible. They narrow its authority and provide software seams that can be verified independently.

Chapter V

SYSTEM DESIGN AND IMPLEMENTATION

V.1 Architectural style

FiPilot is a web application with a React client and a FastAPI backend. The active HTTP boundary is `backend/gateway/main.py`. The gateway composes authentication dependencies, application services, infrastructure adapters, and routers. Route handlers remain thin: they validate transport input, resolve the current user, call an application service, and return a canonical response.

The backend follows four main layers:

1. **gateway**: REST and WebSocket transport, dependency resolution, and exception mapping;
2. **services and orchestrator**: use cases for Resume handling, planning, questions, answers, reports, and interview progression;
3. **shared schemas**: canonical request, response, and domain data contracts; and
4. **infrastructure**: Firebase, document extraction, language, speech, SQLite, and Firestore adapters.

This direction keeps external APIs and storage details outside domain-facing service contracts. It also prevents the compatibility modules under `backend/app` from becoming a second active HTTP application.

V.2 Frontend design

Production routes are declared in `frontend/src/App.tsx`. React Router selects pages; TanStack Query manages server state; Zustand stores client session state where appropriate; and `frontend/src/lib/api.ts` centralizes authenticated API calls. Canonical TypeScript contracts live under `frontend/src/types`.

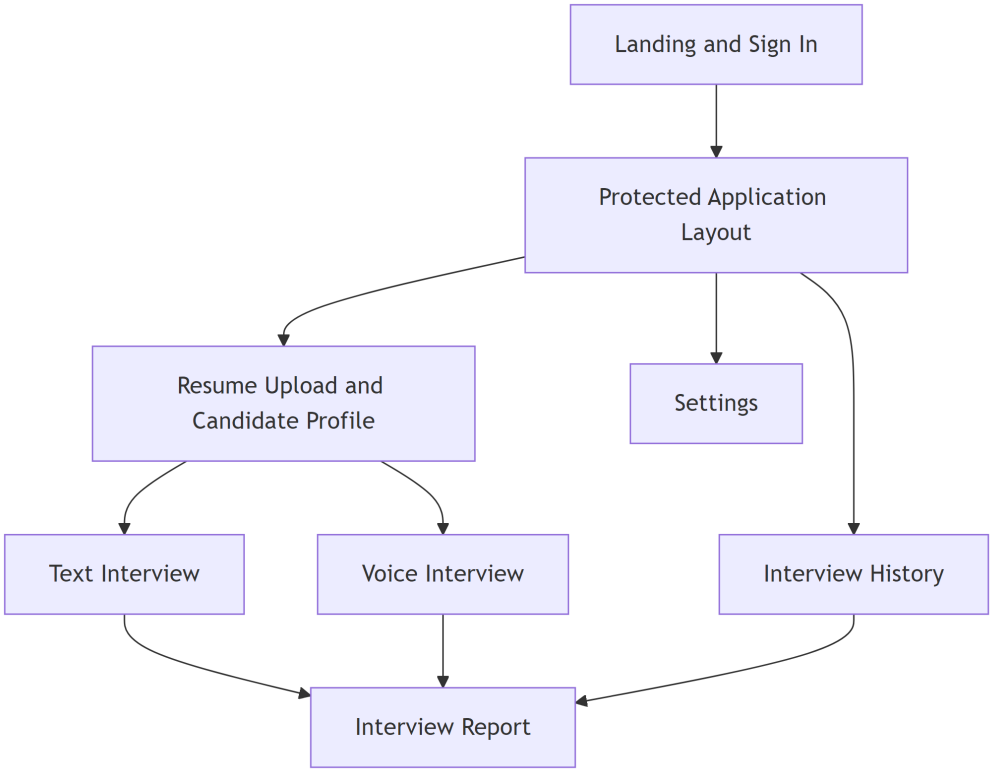

Figure V.1: Production frontend routes and their backend interactions.

The main user-facing areas are Candidate Profile review, text interview, voice interview, report, interview history, and settings. Authentication gates protected routes. The frontend retrieves a Firebase ID token from the current user and sends it as a bearer token. The server, rather than the browser, determines the authenticated user identifier.

V.3 Backend request composition

FastAPI dependencies construct repositories and services from environment settings. The storage implementation can be SQLite or Firestore while exposing the same public repository behavior. Language and speech integrations are also injected through application-facing interfaces. This composition supports local testing with deterministic substitutes without changing route contracts.

Health and readiness routes are separate. The health route indicates that the process is available. The readiness route performs configured dependency checks and can show whether the application is prepared to serve its connected integrations.

V.4 Text interview sequence

The text flow uses REST for preparation, session creation, answers, session reload, and report access. Preparation selects knowledge and creates a plan preview. Session start persists the interview and returns its first question. Each answer request includes the session identifier and an idempotency key.

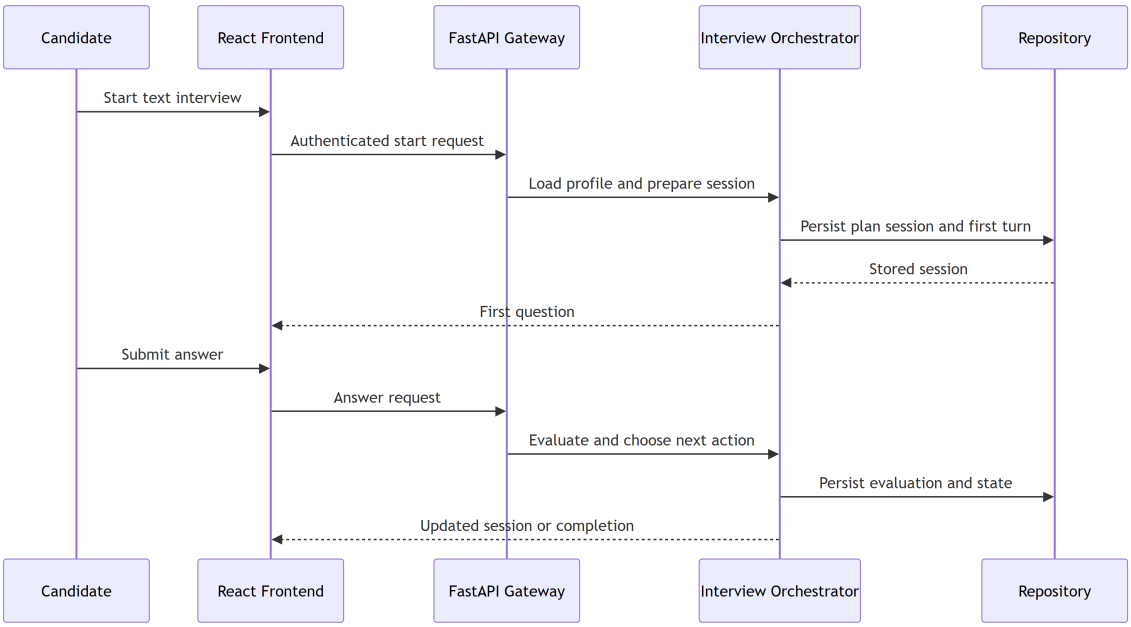

Figure V.2: Text interview request sequence.

On answer submission, the service resolves an owned session, claims the idempotency key, evaluates the answer, decides the next action, persists the updated state, and returns the session. A duplicate request does not create an additional turn. The browser can reload the persisted session after navigation or an interrupted response.

V.5 Voice interview sequence

Voice mode uses an authenticated WebSocket at `/api/v2/voice/interview/:session_id`. Authentication is provided through the negotiated subprotocol; the gateway also checks the request origin and session ownership before accepting an active connection. Binary PCM frames and bounded JSON control messages use the same socket.

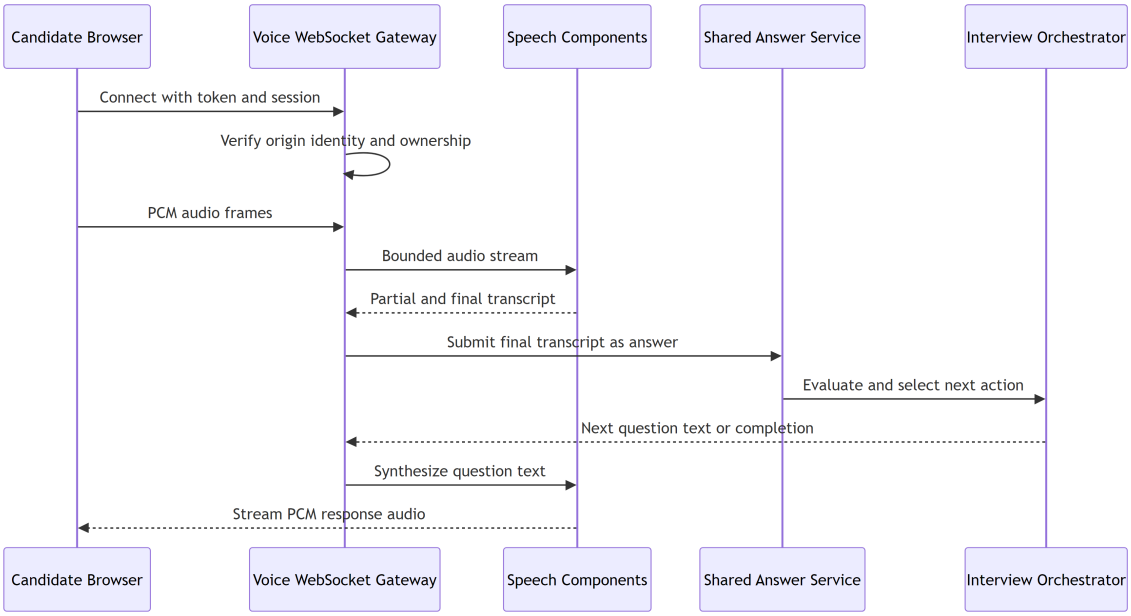

Figure V.3: Voice channel and convergence with the shared answer service.

The voice runtime detects speech, sends audio to the configured speech-to-text component, and receives transcript events. A final transcript becomes the answer input to the same application service used by the REST flow. The gateway streams question and feedback events to the client and can obtain synthesized speech for playback. Connection limits, message limits, and close codes protect the WebSocket boundary.

V.6 Persistence design

The repository contract stores candidates, Resumes, sessions, turns, reports, answer claims, and related status records. SQLite supports local operation and automated tests. Firestore supports document-oriented persistence in a Firebase environment. Both adapters are expected to preserve ownership filters and application invariants.

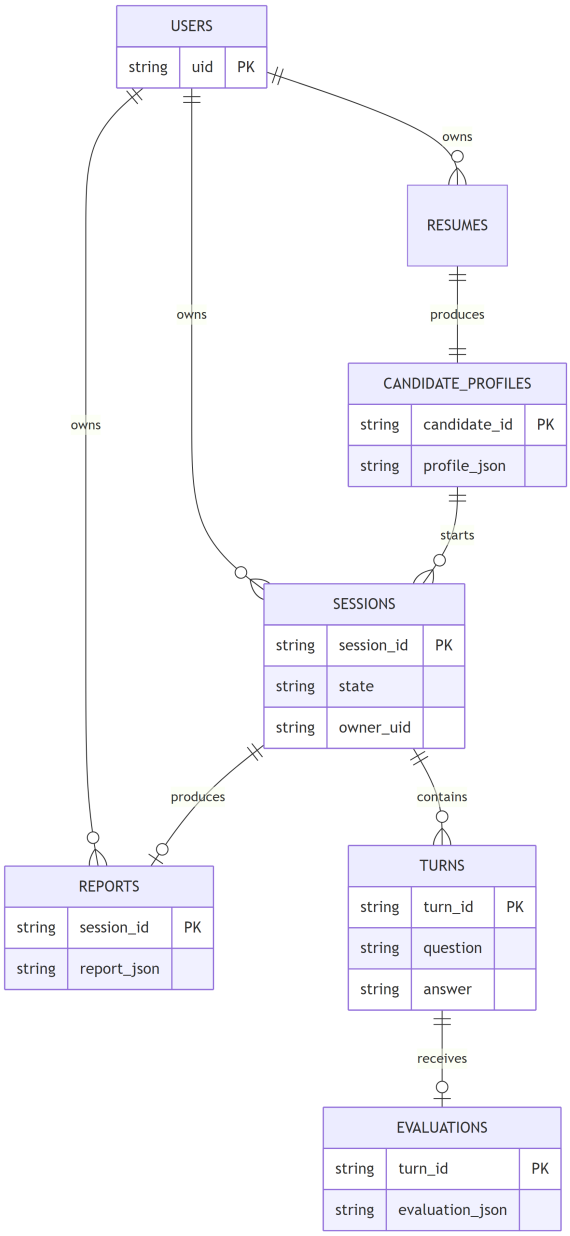

Figure V.4: Principal persisted resources and ownership relationships.

Candidate identifiers and session identifiers are resource locators, not proof of authorization. Repository calls receive the current user's identifier and return the same not-found behavior for missing and foreign resources. This avoids revealing whether another user's resource exists.

V.7 HTTP and WebSocket interfaces

Table V.1: Active application interfaces.

Method	Path	Purpose
GET	`/api/v2/auth/me`	Return the authenticated user contract.
POST	`/api/v2/resume/upload`	Validate, extract, and create candidate resources.
GET	`/api/v2/candidates/:candidate_id/profile`	Return the owned Candidate Profile and readiness data.
POST	`/api/v2/interview/prepare`	Retrieve knowledge and create an Interview Plan.
POST	`/api/v2/interview/start`	Create an interview session and first question.
POST	`/api/v2/interview/:session_id/answer`	Process one idempotent text answer.
GET	`/api/v2/interview/:session_id`	Reload an owned interview session.
POST	`/api/v2/interview/:session_id/report`	Generate and persist a report.
GET	`/api/v2/interview/:session_id/report`	Return the owned report.
GET	`/api/v2/interviews`	Return owned interview history.
WS	`/api/v2/voice/interview/:session_id`	Run the authenticated voice channel.
GET	`/health`, `/ready`	Process health and dependency readiness.

V.8 Authentication and ownership

Firebase Authentication establishes identity. For REST, the frontend sends `Authorization:Bearer <token>`; Firebase Admin verifies the token, and the gateway constructs the current-user dependency. Development authentication can be disabled through local settings, but this mode is not a production authorization mechanism.

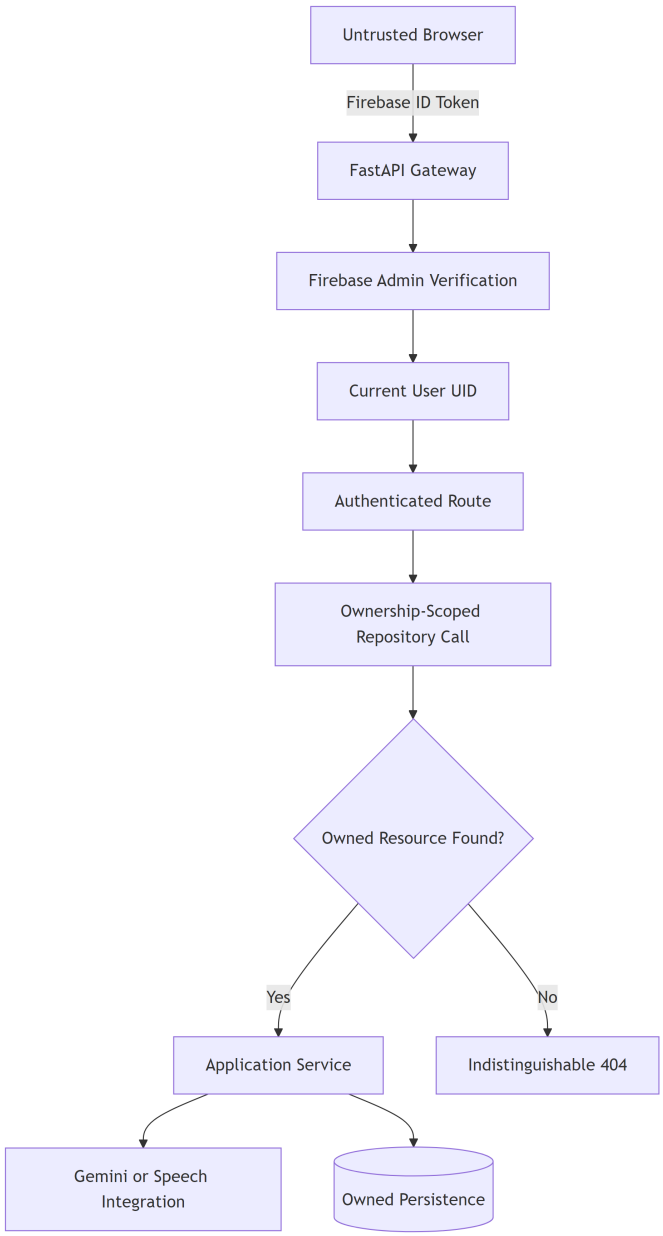

Figure V.5: Identity verification and ownership-scoped resource access.

The backend never accepts an owner identifier as authority from a request body. Profile, upload, interview, status, and report operations are scoped to the verified user. Voice connections apply the same ownership rule before session use.

V.9 Configuration and observability

Runtime settings cover authentication, Firebase, repository selection, Vertex AI, CORS origins, file limits, timeouts, and speech behavior. The repository helper loads `backend/.env.local` by default, or the path supplied through `BACKEND_ENV_FILE`. The open editor file `backend/app/.env` is not automatically loaded by the active helper.

The gateway uses structured logging and application-level exception handlers. Provider errors are translated into stable HTTP responses, while detailed internal messages remain in server logs. Request identifiers and explicit state values support diagnosis without placing application state inside prompt text.

Chapter VI

SOFTWARE REQUIREMENTS SPECIFICATION

VI.1 Product perspective

FiPilot is a browser-based application whose primary workflow begins with an authenticated user and a Resume. The frontend depends on the FastAPI gateway for all authoritative candidate and interview data. The backend depends on Firebase for production identity, a configured Vertex AI project for language operations, a repository adapter for persistence, and speech components for voice mode.

The specification in this chapter documents behavior observable in the connected application. It does not introduce proposed upload replacement, profile-edit mutation, or deployment behavior.

VI.2 Actors and external systems

Table VI.1: Actors and connected systems.

Actor or system	Responsibility
Candidate	Signs in, uploads a Resume, reviews the profile, configures an interview, answers questions, and reads the report.
Firebase Authentication	Issues the client identity token; Firebase Admin verifies it at the backend.
Vertex AI Gemini	Produces schema-constrained profile, plan, question, evaluation, and report content.
SQLite or Firestore	Stores user-scoped application resources through the repository contract.
Speech components	Convert voice audio to transcripts and generated text to audio in voice mode.
Local Knowledge catalog	Supplies reviewable technical topic context to the interview planner.

VI.3 Functional requirements

Table VI.2: Implemented functional requirements.

ID	Requirement
FR-01	The system shall authenticate protected REST and voice operations and resolve the user identity on the server.
FR-02	The system shall accept one PDF or DOCX Resume whose size is at most 10 MB and reject unsupported or structurally invalid files.
FR-03	The system shall extract native document text and use OCR for a PDF when native text is insufficient.
FR-04	The system shall convert accepted Resume text into the canonical Candidate Profile and persist it for the authenticated user.
FR-05	The system shall return the owned Candidate Profile with a strong version tag and structured readiness information.
FR-06	The system shall select relevant local Knowledge records using the candidate and interview configuration.
FR-07	The system shall prepare a structured Interview Plan before an interview begins.
FR-08	The system shall create a persisted text or voice Interview Session and return its current state.
FR-09	The system shall generate a question for the active plan round.
FR-10	The system shall accept a nonblank text answer or a finalized voice transcript and evaluate it through the shared answer service.
FR-11	The system shall prevent the same logical answer submission from producing duplicate turns.
FR-12	The system shall use deterministic workflow rules to continue, follow up, change round, or complete the session.
FR-13	The system shall allow an authenticated owner to reload a session and list interview history.
FR-14	The system shall generate, persist, and return a final report for an owned completed session.
FR-15	The voice interface shall validate origin, authentication, session ownership, message type, and message size before processing audio.
FR-16	Foreign and missing owned resources shall use an indistinguishable not-found response.

VI.4 Primary use cases

VI.4.1 Upload and review a Resume

Precondition: the candidate is authenticated. The candidate selects one PDF or DOCX file. The client sends it to the Resume endpoint. The backend validates and extracts the document, creates a Candidate Profile, persists the candidate resources, and returns the result. The profile page displays the structured fields and readiness issues. If validation, extraction, or provider processing fails, the system returns a structured error and does not present a successful profile.

VI.4.2 Complete a text interview

Precondition: an owned Candidate Profile exists. The candidate selects interview settings and asks the server to prepare the interview. The server selects knowledge and produces a plan. Start creates the session and first question. For each turn, the candidate submits an answer; the service evaluates it, stores the turn, and returns the next state. When the session is complete, the candidate requests and reads the report.

VI.4.3 Complete a voice interview

Precondition: an owned voice-mode session exists and the browser can access a microphone. The client opens the authenticated WebSocket and sends supported control and PCM audio messages. The server produces speech events and transcripts. Final transcripts use the shared answer flow. The candidate receives the question, feedback, and next session state through socket events.

VI.4.4 Review interview history

The authenticated candidate opens the history page. The client requests a paginated list of owned sessions. Selecting a session reloads its stored state or report. The server never uses a client-supplied owner identifier to broaden the query.

VI.5 Data requirements

All persisted candidate, session, and report resources have server-assigned identifiers and an owner. Candidate Profile fields use one canonical naming scheme. Session state contains its mode, plan, current round, turns, status, and timestamps required by the application. Answer claims bind a logical turn to its processing result. Reports are associated with a session and can be retrieved only through ownership-scoped access.

Strong profile versions are returned through the ETag response header. Model-generated data must pass the corresponding Pydantic schema before persistence. Transport schemas reject missing required fields and invalid enum values.

VI.6 Quality requirements

Table VI.3: Quality requirements and implementation mechanisms.

Attribute	Requirement	Mechanism
Security	Protected data is available only to its authenticated owner.	Token verification, current-user dependency, origin check, and ownership-scoped repositories.
Reliability	Retries must not duplicate an answer turn.	Repository-backed idempotency claim and replayed result.
Integrity	AI responses must match application contracts.	Structured generation plus Pydantic validation.
Maintainability	Transport, application logic, and infrastructure remain separable.	Gateway, services, shared schemas, and adapter boundaries.
Portability	Local and Firebase-backed persistence share behavior.	Common repository interface with SQLite and Firestore implementations.
Usability	Users can see progress, errors, current questions, and report output.	Route-specific UI state and structured backend responses.
Accessibility	Core interactions support semantic controls and keyboard operation.	Reusable UI controls, labels, focus treatment, and responsive layouts.
Observability	Failures can be traced without exposing internals to clients.	Request identifiers, structured logs, and stable error responses.

VI.7 Operational limits

The application requires network access for configured Firebase and Vertex AI operations. Voice mode additionally requires browser microphone permission and available speech services. Resume interpretation is limited by document quality and the information present in the file. Generated questions and assessments can contain model errors, so users must treat the report as interview-practice feedback rather than an employment decision.

The repository does not contain a checked-in continuous-integration workflow. Local commands therefore provide the verified software baseline. Environment-specific deployment, live provider quotas, latency at scale, and production availability are outside this report's verified claims.

Chapter VII

SOFTWARE TESTING

VII.1 Test strategy

Testing follows the public boundaries of the application. Schema tests protect data contracts. Service and orchestrator tests exercise use cases with controlled dependencies. Repository contract tests verify both ownership and persistence behavior. API tests send requests through FastAPI. Frontend tests use React Testing Library to exercise rendered user behavior. Build and compilation checks identify integration errors not covered by individual cases.

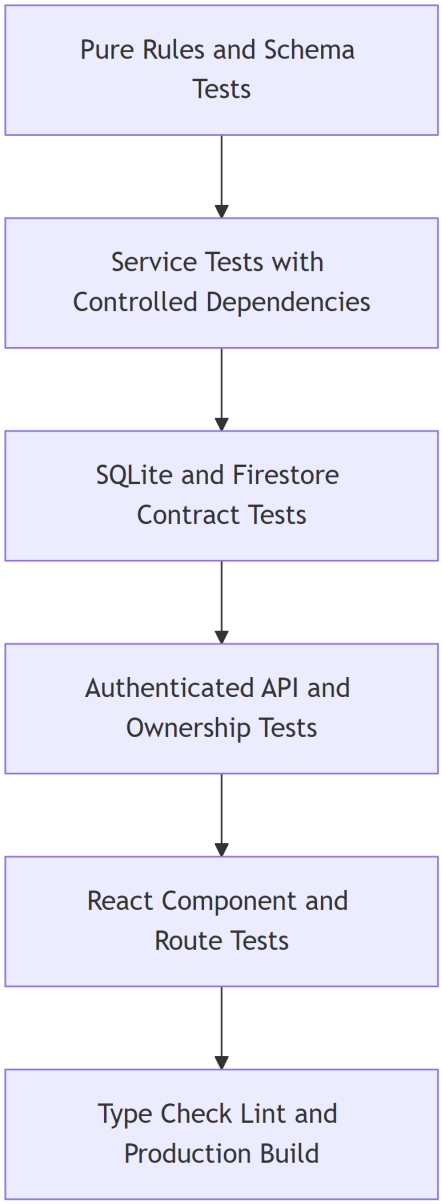

Figure VII.1: Software verification layers used by the project.

The project uses deterministic substitutes for language, authentication, and storage dependencies where a test is intended to verify application behavior. This keeps assertions focused on request contracts, orchestration, error handling, and persistence rather than external provider variability.

VII.2 Backend verification

Backend tests are located in `backend/app/tests`. They cover authentication, ownership, Resume upload, document extraction, repository behavior, Candidate Profile responses, interview preparation and start, answer idempotency, report access, history, voice protocol behavior, and error translation.

Table VII.1: Representative backend test areas.

Area	Verified behavior
Authentication	Missing, malformed, invalid, and accepted identity tokens; local authentication configuration.
Ownership	Owned, foreign, and missing candidates, sessions, reports, and voice connections.
Resume boundary	File type, file signature, maximum size, PDF/DOCX extraction, OCR fallback, and structured errors.
Candidate Profile	Canonical schema, strong ETag, and structured readiness response.
Interview workflow	Preparation, start, state transitions, follow-up decisions, completion, and reload.
Answer submission	Turn claim, concurrent duplicate handling, replay, evaluation failure, and stored result.
Persistence	SQLite and Firestore contract behavior at the application interface.
Voice	Authentication, origin, control messages, binary audio limits, transcript flow, and disconnect behavior.
Reports and history	Generation, retrieval, pagination, not-found behavior, and ownership.

Python compilation is also run over the active backend packages: `core`, `gateway`, `infrastructure`, `orchestrator`, `services`, `shared`, and `speech_service`. The repository does not configure a separate Python static type checker or linter, so this report does not claim such a result.

VII.3 Frontend verification

Frontend tests cover route access, Firebase token handling, API error presentation, upload interaction, Candidate Profile rendering, text interview state, voice state, history, reports, and shared UI behavior. TypeScript project compilation checks client contracts. ESLint checks configured source rules. The production Vite build checks bundling and asset generation.

Table VII.2: Frontend verification commands.

Command	Purpose
`npmexectsc---b`	Compile the TypeScript project graph.
`npmrunlint`	Apply the configured ESLint rules.
`npmtest`	Run the complete Vitest and React Testing Library suite.
`npmrunbuild`	Produce the Vite production bundle.

VII.4 Current verification result

The verification run used for this report produced the results in table VII.3. Counts describe the repository at the time of the run and may change when tests are added.

Table VII.3: Current software verification result.

Check	Result	Evidence
Backend pytest collection	Pass	286 tests passed.
Backend Python compilation	Pass	Active backend packages compiled without syntax errors.
Frontend TypeScript compilation	Pass	Project build completed without type errors.
Frontend lint	Pass	Configured lint command completed successfully.
Frontend test suite	Pass	119 tests passed.
Frontend production build	Pass	Vite created the production bundle.

VII.5 Acceptance-oriented scenarios

Table VII.4: Selected acceptance scenarios.

ID	Scenario	Expected result
AT-01	Authenticated user uploads a valid PDF.	Text is extracted, the profile is validated and persisted, and candidate identifiers are returned.
AT-02	User uploads a DOCX with an invalid binary signature.	The request is rejected without invoking profile generation.
AT-03	PDF contains insufficient native text.	The OCR path is invoked and its extraction status is represented in the result.

ID	Scenario	Expected result
AT-04	User requests another user's candidate profile.	The response is the same not-found shape used for a missing profile.
AT-05	Candidate prepares and starts an interview.	Knowledge is selected, a plan is available, and a persisted session returns its first question.
AT-06	Client repeats the same answer turn.	The stored result is replayed and no duplicate turn is created.
AT-07	Voice connection has an invalid token or disallowed origin.	The gateway closes the connection with the documented protocol response.
AT-08	Final transcript is accepted in voice mode.	It is evaluated through the shared answer service and advances the same session state.
AT-09	Owner requests a completed report.	The service returns the report associated with the owned session.
AT-10	History request uses pagination.	Only owned sessions within the requested page are returned with the total count.

VII.6 Testing limitations

The passing software suite establishes behavior for the checked-in test cases; it does not establish human-level assessment accuracy. The project has no checked-in browser end-to-end or visual-regression suite covering every production route and viewport. This report also makes no claim about load capacity, long-duration WebSocket stability, provider availability, broad Resume populations, or comparative model quality. Those areas require operational or user-study evidence beyond the current application suite.

Chapter VIII

DELIVERABLES AND USER GUIDE

VIII.1 Project deliverables

Table VIII.1: Repository deliverables.

Deliverable	Location and content
Frontend application	`frontend/`: React pages, components, client contracts, API client, styles, and tests.
Backend application	`backend/gateway`, `backend/services`, `backend/orchestrator`, `backend/infrastructure`, and `backend/shared`.
Backend tests	`backend/app/tests`: application boundary and regression coverage.
Knowledge catalog	`backend/services/interview_knowledge`: local catalog, level guide, loader, and retriever.
Speech service	`backend/speech_service`: optional speech runtime used by the voice integration.
Architecture and development documents	`docs/`, `CONTEXT.md`, and repository instructions.
Project report	`report/Final_report.pdf` with LaTeX sources, diagrams, and build scripts.

VIII.2 Local prerequisites

The supported local workflow uses Python 3.12, Node.js with npm, and PowerShell. Firebase and Vertex AI settings are required for connected provider behavior. Voice mode may require the optional speech-service environment described in `docs/local-development.md`.

VIII.3 Installation

From the repository root, create the backend environment and install dependencies:

Listing VIII.1: Backend dependency installation.

```
cd backend
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
```

Install the frontend dependencies:

Listing VIII.2: Frontend dependency installation.

```
cd ..\frontend
npm ci
```

VIII.4 Environment configuration

Create local backend settings in `backend/.env.local`, or point `BACKEND_ENV_FILE` to an explicit settings file before using the repository helper. Configure authentication, Firebase project details, repository selection, Vertex AI location and model settings, allowed frontend origins, and speech settings required by the chosen mode. Secrets must not be committed.

The legacy path `backend/app/.env` is not loaded automatically by `scripts/run_backend.ps1`. Editing that file alone does not change the active gateway configuration.

The frontend Firebase values are supplied through its Vite environment settings. The browser and backend projects must refer to compatible Firebase authentication configuration.

VIII.5 Starting the application

Run the backend from `backend/`:

Listing VIII.3: Start the active FastAPI gateway.

```
python -m uvicorn gateway.main:app --reload `
  --host 127.0.0.1 --port 8000
```

Alternatively, from the repository root:

Listing VIII.4: Start the backend with the repository helper.

```
powershell -ExecutionPolicy Bypass `
  -File .\scripts\run_backend.ps1
```

Run the frontend from `frontend/`:

Listing VIII.5: Start the Vite frontend.

```
npm run dev -- --host localhost
```

For voice mode, start the optional speech service from the repository root as documented in `docs/local-development.md`:

Listing VIII.6: Start the optional speech service.

```
powershell -ExecutionPolicy Bypass `
  -File .\scripts\run_speech_service.ps1
```

VIII.6 Candidate workflow

VIII.6.1 Sign in and upload

Open the frontend and sign in with the configured Firebase provider. Choose a single Resume in PDF or DOCX format. The maximum accepted size is 10 MB. Wait for validation, extraction, and profile creation to complete. If the application rejects the file, correct the indicated file type, file structure, size, or provider configuration before retrying.

VIII.6.2 Review the Candidate Profile

Open Candidate Profile review and inspect the name, recent role, specialization, skills, evidence, projects, experiences, and education displayed from the server response. Readiness issues identify missing information required by the profile-readiness rules. The displayed strong version belongs to the server resource.

VIII.6.3 Configure and start a text interview

Choose the interview settings available in the interface. Preparation selects Knowledge records and creates the Interview Plan. Start creates a new session. Read the current question, submit one answer, and wait for the returned feedback and next question. Avoid sending the same turn from multiple tabs; if a network retry occurs, the server's answer claim protects the stored turn.

VIII.6.4 Use voice mode

Grant browser microphone permission and open the voice interview route for the session. The status display indicates connection and speech state. Speak after the current question. The application

sends audio, shows transcripts and feedback, and advances through the shared session workflow. If the connection closes, use the displayed error and server state to recover rather than starting unrelated submissions.

VIII.6.5 Read history and reports

The History page lists owned interview sessions. Open a completed session and request its report. Review the summary, strengths, improvement areas, and recommendations as practice guidance. Model-generated evaluation is not an employment decision or a verified measure of professional competence.

VIII.7 Developer verification

Before handing off a change, run the backend tests and compilation from `backend/`:

Listing VIII.7: Backend verification.

```
python -m pytest
python -m compileall -q core gateway infrastructure `
  orchestrator services shared speech_service
```

Run the frontend checks from `frontend/`:

Listing VIII.8: Frontend verification.

```
npm exec tsc -- -b
npm run lint
npm test
npm run build
```

The repository does not include a checked-in continuous-integration workflow, so these local checks are required for the current project baseline.

Chapter IX

CONCLUSION

IX.1 Achieved result

FiPilot implements an authenticated CV-to-interview workflow across a React frontend and a FastAPI backend. It validates and extracts PDF or DOCX Resumes, uses OCR when a PDF lacks sufficient text, creates a canonical Candidate Profile, selects technical context from a local Knowledge catalog, prepares an Interview Plan, generates questions, evaluates text or transcribed voice answers, applies deterministic session decisions, persists interview history, and produces a final report.

The main architectural result is not a single prompt. It is the separation of responsibilities around probabilistic language operations. Schemas define accepted output. Repository state defines the interview. Firebase identity defines the user. Lexical retrieval defines the selected technical context. The decision service defines progression. REST and WebSocket channels converge before answer evaluation. These boundaries make the application understandable and testable as software.

IX.2 Technical contributions

The project contributes a cohesive implementation in the following areas:

- a guarded document-ingestion path for PDF and DOCX with OCR fallback;
- one Candidate Profile contract shared by Resume review and interview services;
- a reviewable, deterministic local knowledge-selection mechanism;
- schema-constrained language operations for profile, plan, question, evaluation, and report data;
- persisted multi-turn orchestration with idempotent answer submission;
- a shared answer boundary for text and voice interaction;
- ownership-scoped access implemented at gateway and repository seams; and

- interchangeable SQLite and Firestore persistence behind one application contract.

IX.3 Verified software status

The current repository verification passed 286 backend tests, backend Python compilation, frontend TypeScript compilation, configured lint, 119 frontend tests, and the production frontend build. These results support the implemented software contracts documented in this report. They do not establish broad Resume interpretation accuracy, speech accuracy, model-comparison quality, production scale, or service availability.

IX.4 Known limitations

Resume extraction remains sensitive to unusual layout and scan quality. Automated evaluation can be wrong or incomplete and must remain advisory. Profile readiness is returned by the backend, but the current start route does not yet use it as an authoritative rejection. Session creation and report-profile consistency do not yet provide the complete immutable profile snapshot behavior required for strong historical reproducibility. The active Resume route also does not provide the full replacement and persisted upload-idempotency lifecycle described in broader design documents.

Voice operation depends on microphone quality, connection stability, and configured speech services. Live provider status and deployment behavior vary by environment and were not used as evidence for this report. The repository has no checked-in browser end-to-end, visual-regression, load, or production-availability suite.

IX.5 Overall assessment

Within these limits, FiPilot is a complete application-level demonstration of governed language-service integration. It connects Resume evidence, curated technical knowledge, a persisted interview workflow, two interaction channels, and user-owned reports without assigning authentication or workflow authority to model output. The result is a credible foundation for interview practice and a concrete software-engineering case study in applying AI services through explicit contracts.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] L. Zheng, W.-L. Chiang, Y. Sheng, et al., “Judging llm-as-a-judge with mt-bench and chatbot arena,” *arXiv preprint arXiv:2306.05685*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

Appendix A

RUNTIME CONTRACT REFERENCE

A.1 Canonical profile structure

Table A.1: Candidate Profile nested structure.

Structure	Canonical fields
Candidate Profile	`name`, `years_experience`, `recent_role`, `specialization`, `skills`, `skill_evidence`, `projects`, `experiences`, `education`.
Project	`name`, `description`, `technologies`, `role`.
Experience	`company`, `title`, `start_date`, `end_date`, `description`, `technologies`.
Structured education	`institution`, `degree`, `field_of_study`, `start_date`, `end_date`.
Skill evidence	`skill`, `evidence`, and server-issued `evidence_id` for an existing entry.

A.2 Runtime source index

Table A.2: Principal source locations supporting the report.

Location	Responsibility
`backend/gateway/main.py`	Active FastAPI application and router composition.
`backend/gateway/api`	REST and WebSocket transport endpoints.
`backend/core/dependencies.py`	Authentication and application dependency composition.
`backend/shared/schemas`	Canonical backend transport and domain contracts.

Location	Responsibility
`backend/services/profile_scanner`	Resume-to-profile language operation.
`backend/services/interview_knowledge`	Local catalog loading and lexical retrieval.
`backend/services/interview_planner`	Structured Interview Plan generation.
`backend/services/question_generator`	Turn-level question generation.
`backend/services/answer_evaluator`	Structured answer evaluation.
`backend/services/interview_answer_service.py`	Shared idempotent answer use case.
`backend/orchestrator`	Conversation progression and deterministic decisions.
`backend/services/report_generator`	Report generation and retrieval service.
`backend/infrastructure/documents`	PDF, DOCX, and OCR extraction.
`backend/infrastructure/repositories`	SQLite and Firestore persistence adapters.
`backend/infrastructure/llm`	Vertex AI Gemini integration.
`backend/infrastructure/speech`	Speech integration adapters.
`frontend/src/App.tsx`	Production route declaration.
`frontend/src/lib/api.ts`	Centralized backend API client.
`frontend/src/types`	Canonical frontend contracts.

A.3 Report build

The report sources are in `report/`. Mermaid sources under `report/diagrams` are rendered to high-resolution PNG files before XeLaTeX compilation. Each report figure is placed at a readable page width with a bounded height and an intervening float barrier, which prevents later text or figures from overlaying it.

From the repository root, build the report with:

Listing A.1: Project report build command.

```
powershell -ExecutionPolicy Bypass `
  -File .\report\scripts\build_report.ps1
```

The build performs diagram rendering, XeLaTeX and Biber passes, and log checks for unresolved references and material page overflow.